

Principles of Programming Languages

Big-step Structural Operational Semantics

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Structural Operational Semantics

Big-step SOS

- Configurations

- Arithmetic expressions

- Boolean expressions

- Statements

Structural Operational Semantics

A language designer should understand the existing design approaches:

- ▶ Big-step structural operational semantics
- ▶ Small-step structural operational semantics
- ▶ Denotational Semantics
- ▶ Modular operational semantics
- ▶ Reduction semantics with evaluation contexts
- ▶ Abstract Machines, the chemical abstract machine
- ▶ Axiomatic semantics

Slide credits: Grigore Roşu, PhD., Full Professor - UIUC

Structural Operational Semantics

A language designer should understand the existing design approaches:

- ▶ Big-step structural operational semantics
- ▶ Small-step structural operational semantics
- ▶ Denotational Semantics (partially covered in the previous lecture)
- ▶ Modular operational semantics
- ▶ Reduction semantics with evaluation contexts
- ▶ Abstract Machines, the chemical abstract machine
- ▶ Axiomatic semantics

Structural Operational Semantics (SOS)

- ▶ SOS was proposed by Plotkin in 1981
- ▶ Simple framework to describe the behaviour of the language constructs as inference rules
- ▶ Rules specify transitions between program configurations
- ▶ Configurations are tuples of various kinds of data structures (e.g., trees, sets, lists)
 - ▶ capture various components of program states (environment, memory, stacks, registers, etc.)

Big-step Structural Operational Semantics

- ▶ Probably the most natural way of defining structural operational semantics
- ▶ A.k.a. *natural semantics*, *relational semantics*, *evaluation semantics*
- ▶ Big-step SOS for a PL: a set of inference rules

The language that we use to illustrate SOS is IMP with:

- ▶ Arithmetic expressions
- ▶ Boolean Expressions
- ▶ Statements: assignments, (possibly empty) blocks/sequences of statements, decisionals (if-then-else), loops (while)

Outline

Structural Operational Semantics

Big-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Big-step SOS configurations

Configurations are tuples: $\langle \text{code}, \text{environment}, \dots \rangle$.

They hold various information needed to execute a program:

- ▶ the code that needs to be executed;
- ▶ the current environment;
- ▶ the program stack;
- ▶ the current program counter;
- ▶ ...

Big-step SOS configurations

Configurations are tuples: $\langle \text{code}, \text{environment}, \dots \rangle$.

They hold various information needed to execute a program:

- ▶ the code that needs to be executed;
- ▶ the current environment;
- ▶ the program stack;
- ▶ the current program counter;
- ▶ ...

IMP configurations

For instance, IMP configurations can store the following information:

- ▶ $\langle A, E \rangle$, where A has type `AExp` and E has type `Env`;
- ▶ $\langle N \rangle$, where N has type `nat`;
- ▶ $\langle B, E \rangle$, where B has type `BExp` and E has type `Env`;
- ▶ $\langle B \rangle$, where A has type `bool`;
- ▶ $\langle E \rangle$, where E has type `Env`;
- ▶ $\langle S, E \rangle$, where S has type `Stmt` and E has type `Env`;
- ▶ $\langle S \rangle$, where S has type `Stmt`.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Sequents

Big-step SOS sequents: relations over configurations of the form $C \Downarrow R$, where

- ▶ C is a configuration
- ▶ R is a *result configuration* that is obtained after the complete evaluation of C .

Result configurations are also called *irreducible configurations*.

Informally, a sequent says that a configuration C *evaluates/executes/transitions* to a configuration R .

Examples: $\langle 1, \sigma \rangle \Downarrow \langle 1 \rangle$, $\langle 1 + 2 \rangle \Downarrow \langle 3 \rangle$, $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$.

Rule schemata

The *Big-step SOS rules* have this form:

$$\frac{C_1 \Downarrow R_1 \quad C_2 \Downarrow R_2 \quad \dots \quad C_n \Downarrow R_n}{C_0 \Downarrow R_0},$$

where $C_0, C_1, \dots, C_n$ are configurations and $R_0, R_1, \dots, R_n$ are result configurations.

Here is an example of a big-step rule for IMP:

$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + a_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

Rule schemata

The *Big-step SOS rules* have this form:

$$\frac{C_1 \Downarrow R_1 \quad C_2 \Downarrow R_2 \quad \dots \quad C_n \Downarrow R_n}{C_0 \Downarrow R_0},$$

where $C_0, C_1, \dots, C_n$ are configurations and $R_0, R_1, \dots, R_n$ are result configurations.

Here is an example of a big-step rule for IMP:

$$\frac{\langle \mathbf{a}_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle \mathbf{a}_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle \mathbf{a}_1 + \mathbf{a}_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

Outline

Structural Operational Semantics

Big-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Big-step rules for IMP – 1: arithmetic expressions

CONST: $\langle i, \sigma \rangle \Downarrow \langle i \rangle$

LOOKUP: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$ if $\sigma(x) \neq \perp$

ADD:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + a_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

MUL:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 * a_2, \sigma \rangle \Downarrow \langle i_1 *_{\text{nat}} i_2 \rangle}$$

Big-step rules for IMP – 1: arithmetic expressions

CONST: $\langle i, \sigma \rangle \Downarrow \langle i \rangle$

LOOKUP: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$ if $\sigma(x) \neq \perp$

ADD:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + a_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

MUL:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 * a_2, \sigma \rangle \Downarrow \langle i_1 *_{\text{nat}} i_2 \rangle}$$

Big-step rules for IMP – 1: arithmetic expressions

CONST: $\langle i, \sigma \rangle \Downarrow \langle i \rangle$

LOOKUP: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$ if $\sigma(x) \neq \perp$

ADD:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + a_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

MUL:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 * a_2, \sigma \rangle \Downarrow \langle i_1 *_{\text{nat}} i_2 \rangle}$$

Big-step rules for IMP – 1: arithmetic expressions

CONST: $\langle i, \sigma \rangle \Downarrow \langle i \rangle$

LOOKUP: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$ if $\sigma(x) \neq \perp$

ADD:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + a_2, \sigma \rangle \Downarrow \langle i_1 +_{\text{nat}} i_2 \rangle}$$

MUL:
$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 * a_2, \sigma \rangle \Downarrow \langle i_1 *_{\text{nat}} i_2 \rangle}$$

rule schema vs. rule?

The following *rule*:

$$\frac{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle \quad \langle n, \sigma \rangle \Downarrow \langle 10 \rangle}{\langle 2 +' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10 \rangle}$$

is a an instance of the *rule schema*:

$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 +' a_2, \sigma \rangle \Downarrow \langle i_1 +_{nat} i_2 \rangle} \text{ BIGSTEP-ADD.}$$

On the other hand, this is not an instance of BIGSTEP-ADD:

$$\frac{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle \quad \langle n, \sigma \rangle \Downarrow \langle 10 \rangle}{\langle 2 +' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10, \sigma \rangle} ?$$

because $\langle 2 +' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10, \sigma \rangle$ is **not** an instance of the conclusion of BIGSTEP-ADD.

rule schema vs. rule?

The following *rule*:

$$\frac{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle \quad \langle n, \sigma \rangle \Downarrow \langle 10 \rangle}{\langle 2 + ' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10 \rangle}$$

is a an instance of the *rule schema*:

$$\frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 + ' a_2, \sigma \rangle \Downarrow \langle i_1 +_{nat} i_2 \rangle} \text{ BIGSTEP-ADD.}$$

On the other hand, this is not an instance of BIGSTEP-ADD:

$$\frac{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle \quad \langle n, \sigma \rangle \Downarrow \langle 10 \rangle}{\langle 2 + ' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10, \sigma \rangle} ?$$

because $\langle 2 + ' n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10, \sigma \rangle$ is **not** an instance of the conclusion of BIGSTEP-ADD.

Derivation

- ▶ A derivation example where $\sigma(n) = 10$:

$$\frac{\frac{\cdot}{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle} \text{CONST} \quad \frac{\cdot}{\langle n, \sigma \rangle \Downarrow \langle 10 \rangle} \text{LOOKUP}}{\langle 2 + n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10 \rangle} \text{ADD}$$

Outline

Structural Operational Semantics

Big-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Big-step rules for IMP – 2: boolean expressions

TRUE: $\langle \text{btrue}, \sigma \rangle \Downarrow \langle \text{true} \rangle$

FALSE: $\langle \text{bfalse}, \sigma \rangle \Downarrow \langle \text{false} \rangle$

NOTTRUE:
$$\frac{\langle b, \sigma \rangle \Downarrow \langle \text{true} \rangle}{\langle !b, \sigma \rangle \Downarrow \langle \text{false} \rangle}$$

NOTFALSE:
$$\frac{\langle b, \sigma \rangle \Downarrow \langle \text{false} \rangle}{\langle !b, \sigma \rangle \Downarrow \langle \text{true} \rangle}$$

Big-step rules for IMP – 2: boolean expressions

$$\text{ANDTRUE:} \quad \frac{\langle b_1, \sigma \rangle \Downarrow \langle \text{true} \rangle \quad \langle b_2, \sigma \rangle \Downarrow \langle b \rangle}{\langle b_1 \text{ and } b_2, \sigma \rangle \Downarrow \langle b \rangle}$$

$$\text{ANDFALSE:} \quad \frac{\langle b_1, \sigma \rangle \Downarrow \langle \text{false} \rangle}{\langle b_1 \text{ and } b_2, \sigma \rangle \Downarrow \langle \text{false} \rangle}$$

$$\text{LT:} \quad \frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 < a_2, \sigma \rangle \Downarrow \langle i_1 <_{\text{nat}} i_2 \rangle}$$

$$\text{GT:} \quad \frac{\langle a_1, \sigma \rangle \Downarrow \langle i_1 \rangle \quad \langle a_2, \sigma \rangle \Downarrow \langle i_2 \rangle}{\langle a_1 > a_2, \sigma \rangle \Downarrow \langle i_1 >_{\text{nat}} i_2 \rangle}$$

Derivation example

A derivation tree for $\langle 2 + n < 10, \sigma \rangle \Downarrow \langle (2 +_{nat} 10) <_{nat} 10 \rangle$:

$$\begin{array}{c}
 \frac{\frac{\cdot}{\langle 2, \sigma \rangle \Downarrow \langle 2 \rangle} \text{CONST} \quad \frac{\cdot}{\langle n, \sigma \rangle \Downarrow \langle 10 \rangle} \text{LOOKUP}}{\langle 2 + n, \sigma \rangle \Downarrow \langle 2 +_{nat} 10 \rangle} \text{ADD} \quad \frac{\cdot}{\langle 10, \sigma \rangle \Downarrow \langle 10 \rangle} \text{CONST} \\
 \hline
 \langle 2 + n < 10, \sigma_1 \rangle \Downarrow \langle (2 +_{nat} 10) <_{nat} 10 \rangle \text{LT}
 \end{array}$$

Outline

Structural Operational Semantics

Big-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Big-step rules for IMP – 3: statements

ASSIGN:
$$\frac{\langle a, \sigma \rangle \Downarrow \langle i \rangle}{\langle x ::= a, \sigma \rangle \Downarrow \langle \sigma[i/x] \rangle} \quad \text{if } \sigma(x) \neq \perp$$

SEQ:
$$\frac{\langle s_1, \sigma \rangle \Downarrow \langle \sigma_1 \rangle \quad \langle s_2, \sigma_1 \rangle \Downarrow \langle \sigma_2 \rangle}{\langle s_1 ;; s_2, \sigma \rangle \Downarrow \langle \sigma_2 \rangle}$$

SKIP:
$$\langle \text{skip}, \sigma \rangle \Downarrow \langle \sigma \rangle$$

WHILEFALSE:
$$\frac{\langle b, \sigma \rangle \Downarrow \langle \text{false} \rangle}{\langle \text{while } b \text{ s}, \sigma \rangle \Downarrow \langle \sigma \rangle}$$

WHILETRUE:
$$\frac{\langle b, \sigma \rangle \Downarrow \langle \text{true} \rangle \quad \langle s ;; \text{while } b \text{ s}, \sigma \rangle \Downarrow \langle \sigma' \rangle}{\langle \text{while } b \text{ s}, \sigma \rangle \Downarrow \langle \sigma' \rangle}$$

Exercise

Write a derivation for the sequent:

$$\langle i ::= 0; ; \text{while } i < 1 \ (i ::= i + 1), \sigma \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle.$$

The derivation is shown in the next slides

Derivation

$$\frac{\frac{\cdot}{\langle i := 0, \sigma \rangle \Downarrow \langle \sigma[0/i] \rangle} \text{ ASSIGN} \quad A}{\langle i := 0; ; \text{while } i < 1 (i := i + 1), \sigma \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \text{ SEQ}$$

Derivation for A

$$\frac{
 \frac{
 \overline{\langle i, \sigma[0/i] \rangle \Downarrow \langle 0 \rangle} \text{ LOOKUP}
 \quad
 \frac{
 \overline{\langle 1, \sigma[0/i] \rangle \Downarrow \langle 1 \rangle} \text{ CONST}
 }{
 \text{LT}
 }
 }{
 \langle i < 1, \sigma[0/i] \rangle \Downarrow \langle \text{true} \rangle
 }
 }{
 \langle \text{while } i < 1 (i := i+1), \sigma[0/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle
 }
 \text{ WHILETRUE}$$

Derivation for B

$$\begin{array}{c}
 \frac{\cdot}{\langle i, \sigma[0/i] \rangle \Downarrow \langle 0 \rangle} \text{ LOOKUP} \quad \frac{\cdot}{\langle 1, \sigma[0/i] \rangle \Downarrow \langle 1 \rangle} \text{ CONST} \\
 \hline
 \frac{\langle i+1 \rangle \Downarrow \langle 1 \rangle}{\langle i := i+1 \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \text{ ASSIGN} \quad \text{C} \\
 \hline
 \langle i := i+1; ; \text{while } i < 1 (i := i+1), \sigma[0/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle \text{ SEQ}
 \end{array}$$

Derivation for C

$$\frac{\frac{\cdot}{\langle i, (\sigma[0/i])[1/i] \rangle \Downarrow \langle 1 \rangle} \text{LOOKUP} \quad \frac{\cdot}{\langle 1, (\sigma[0/i])[1/i] \rangle \Downarrow \langle 1 \rangle} \text{CONST}}{\langle i < 1, (\sigma[0/i])[1/i] \rangle \Downarrow \langle \text{false} \rangle} \text{LT} \\
 \frac{\langle \text{while } i < 1 (i := i+1), (\sigma[0/i])[1/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle}{} \text{WHILEFALSE}$$

Complete derivation

The assembled derivation (you're not supposed to read this):

$$\begin{array}{c}
 \frac{}{\langle i, \sigma[0/i] \rangle \Downarrow \langle 0 \rangle} \quad \frac{}{\langle 1, \sigma[0/i] \rangle \Downarrow \langle 1 \rangle} \quad \frac{}{\langle i, (\sigma[0/i])[1/i] \rangle \Downarrow \langle 1 \rangle} \quad \frac{}{\langle 1, (\sigma[0/i])[1/i] \rangle \Downarrow \langle 1 \rangle} \\
 \frac{}{\langle i, \sigma[0/i] \rangle \Downarrow \langle 0 \rangle} \quad \frac{}{\langle 1, \sigma[0/i] \rangle \Downarrow \langle 1 \rangle} \quad \frac{\langle i+1 \rangle \Downarrow \langle 1 \rangle}{\langle i := i+1 \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \quad \frac{\langle i+1, (\sigma[0/i])[1/i] \rangle \Downarrow \langle false \rangle}{\langle while \ i < 1 \ (i := i+1), (\sigma[0/i])[1/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \\
 \frac{}{\langle i < 1, \sigma[0/i] \rangle \Downarrow \langle true \rangle} \quad \frac{}{\langle i := i+1; while \ i < 2 \ (i := i+1), \sigma[0/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \\
 \frac{\langle i := 0, \sigma \rangle \Downarrow \langle \sigma[0/i] \rangle}{\langle i := 0; while \ i < 1 \ (i := i+1), \sigma \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle} \quad \frac{}{\langle while \ i < 1 \ (i := i+1), \sigma[0/i] \rangle \Downarrow \langle (\sigma[0/i])[1/i] \rangle}
 \end{array}$$

Bibliography

- ▶ Chapter Simple Imperative Programs, Section: Evaluation as relation in **Software Foundations - Volume 1**, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey
<https://softwarefoundations.cis.upenn.edu/lf-current/Imp.html#lab368>